

Assignment 5 — Notebook 03: Huấn Luyện & Đánh Giá CNN Keras (NEW DATASETS)

Huấn luyện 2 dạng mô hình CNN (3-layer và 5-layer) trên 3 bộ dữ liệu:

1. **Fashion-MNIST** (Ảnh thời trang Zalando 10 lớp)
2. **SVHN** (Ảnh màu biển số nhà đường phố Google Street View)
3. **CDC Diabetes** (1D-CNN trên tập y tế tăng cường 513.703 dòng)

```
In [1]: # Cell 1: Khai báo môi trường Keras
import os
os.environ['KMP_DUPLICATE_LIB_OK'] = 'TRUE'
import json, time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix, accuracy_score, f1_score

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

QUICK_DEMO = False
print(f'TensorFlow: {tf.__version__} | Keras: {keras.__version__}')
```

TensorFlow: 2.22.0-rc0 | Keras: 3.16.0.dev2026092704

PHẦN A: HUẤN LUYỆN TRÊN BỘ DỮ LIỆU FASHION-MNIST

```
In [2]: # Cell 2: Nạp Fashion-MNIST
f_data = np.load('data/fashion_mnist/fashion_mnist_data.npz')
x_tr_f = np.expand_dims(f_data['x_train'].astype(np.float32) / 255.0, -1)
y_tr_f = f_data['y_train']
x_te_f = np.expand_dims(f_data['x_test'].astype(np.float32) / 255.0, -1)
y_te_f = f_data['y_test']
fashion_classes = ['T-shirt', 'Trouser', 'Pullover', 'Dress', 'Coat', 'Sandal', 'Sh
print(f'Fashion-MNIST shape: {x_tr_f.shape}')
```

Fashion-MNIST shape: (60000, 28, 28, 1)

```
In [3]: # Cell 3: Định nghĩa Keras 3-Layer và 5-Layer cho Fashion-MNIST
m3_f = keras.Sequential([
    layers.Input(shape=(28, 28, 1)),
    layers.Conv2D(32, (3, 3), activation='relu', padding='same'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
    layers.MaxPooling2D((2, 2)),
```

```

        layers.Flatten(),
        layers.Dense(10, activation='softmax')
    ], name='Keras_Fashion_3Layer')
m3_f.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['ac

m5_f = keras.Sequential([
    layers.Input(shape=(28, 28, 1)),
    layers.Conv2D(32, (3, 3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.Conv2D(32, (3, 3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25),
    layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(0.5),
    layers.Dense(10, activation='softmax')
], name='Keras_Fashion_5Layer')
m5_f.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['ac
print(f'M3 Params: {m3_f.count_params():,} | M5 Params: {m5_f.count_params():,}')

```

M3 Params: 50,186 | M5 Params: 469,098

```

In [4]: # Cell 4: Huấn Luyện và đánh giá trên Fashion-MNIST
ep_f = 2 if QUICK_DEMO else 10
print(f'Huấn luyện Fashion-MNIST 3-Layer ({ep_f} epochs)...')
h3_f = m3_f.fit(x_tr_f, y_tr_f, validation_split=0.1, epochs=ep_f, batch_size=128,
print(f'\nHuấn luyện Fashion-MNIST 5-Layer ({ep_f} epochs)...')
h5_f = m5_f.fit(x_tr_f, y_tr_f, validation_split=0.1, epochs=ep_f, batch_size=128,

_, acc3_f = m3_f.evaluate(x_te_f, y_te_f, verbose=0)
_, acc5_f = m5_f.evaluate(x_te_f, y_te_f, verbose=0)
print(f'\n[KẾT QUẢ TEST FASHION-MNIST] 3-Layer: {acc3_f*100:.2f}% | 5-Layer: {acc5_

```

Huấn luyện Fashion-MNIST 3-Layer (10 epochs)...

```
Epoch 1/10 - 16s - loss: 0.5550 - accuracy: 0.8043 - val_loss: 0.4031 - val_accuracy: 0.8512
Epoch 2/10 - 16s - loss: 0.3548 - accuracy: 0.8756 - val_loss: 0.3437 - val_accuracy: 0.8777
Epoch 3/10 - 16s - loss: 0.3121 - accuracy: 0.8903 - val_loss: 0.3160 - val_accuracy: 0.8877
Epoch 4/10 - 16s - loss: 0.2861 - accuracy: 0.8994 - val_loss: 0.2947 - val_accuracy: 0.8933
Epoch 5/10 - 16s - loss: 0.2669 - accuracy: 0.9066 - val_loss: 0.2800 - val_accuracy: 0.8990
Epoch 6/10 - 16s - loss: 0.2512 - accuracy: 0.9116 - val_loss: 0.2717 - val_accuracy: 0.9005
Epoch 7/10 - 16s - loss: 0.2373 - accuracy: 0.9158 - val_loss: 0.2638 - val_accuracy: 0.9028
Epoch 8/10 - 16s - loss: 0.2246 - accuracy: 0.9198 - val_loss: 0.2563 - val_accuracy: 0.9065
Epoch 9/10 - 16s - loss: 0.2131 - accuracy: 0.9242 - val_loss: 0.2514 - val_accuracy: 0.9077
Epoch 10/10 - 16s - loss: 0.2021 - accuracy: 0.9285 - val_loss: 0.2478 - val_accuracy: 0.9107
```

Huấn luyện Fashion-MNIST 5-Layer (10 epochs)...

```
Epoch 1/10 - 129s - loss: 0.5013 - accuracy: 0.8282 - val_loss: 0.9373 - val_accuracy: 0.7118
Epoch 2/10 - 129s - loss: 0.3176 - accuracy: 0.8866 - val_loss: 0.2531 - val_accuracy: 0.9075
Epoch 3/10 - 129s - loss: 0.2709 - accuracy: 0.9036 - val_loss: 0.2209 - val_accuracy: 0.9197
Epoch 4/10 - 129s - loss: 0.2427 - accuracy: 0.9134 - val_loss: 0.2499 - val_accuracy: 0.9070
Epoch 5/10 - 129s - loss: 0.2266 - accuracy: 0.9187 - val_loss: 0.2222 - val_accuracy: 0.9210
Epoch 6/10 - 129s - loss: 0.2131 - accuracy: 0.9238 - val_loss: 0.1920 - val_accuracy: 0.9298
Epoch 7/10 - 129s - loss: 0.2012 - accuracy: 0.9280 - val_loss: 0.1994 - val_accuracy: 0.9295
Epoch 8/10 - 129s - loss: 0.1895 - accuracy: 0.9317 - val_loss: 0.1912 - val_accuracy: 0.9332
Epoch 9/10 - 129s - loss: 0.1848 - accuracy: 0.9322 - val_loss: 0.2223 - val_accuracy: 0.9203
Epoch 10/10 - 129s - loss: 0.1743 - accuracy: 0.9376 - val_loss: 0.1885 - val_accuracy: 0.9322
```

[KẾT QUẢ TEST FASHION-MNIST] 3-Layer: 90.49% | 5-Layer: 92.32%

PHẦN B: HUẤN LUYỆN TRÊN BỘ DỮ LIỆU SVHN

```
In [5]: # Cell 5: Nạp dữ liệu SVHN
svhn_data = np.load('data/svhn/svhn_data.npz')
x_tr_s = svhn_data['x_train'].astype(np.float32) / 255.0
y_tr_s = svhn_data['y_train']
x_te_s = svhn_data['x_test'].astype(np.float32) / 255.0
```

```
y_te_s = svhn_data['y_test']
print(f'SVHN x_train shape: {x_tr_s.shape}')
```

SVHN x_train shape: (73257, 32, 32, 3)

```
In [6]: # Cell 6: Xây dựng và huấn luyện trên SVHN
m3_s = keras.Sequential([
    layers.Input(shape=(32, 32, 3)),
    layers.Conv2D(32, (3, 3), activation='relu', padding='same'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(10, activation='softmax')
])
m3_s.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['ac

m5_s = keras.Sequential([
    layers.Input(shape=(32, 32, 3)),
    layers.Conv2D(32, (3, 3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.Conv2D(32, (3, 3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25),
    layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(0.5),
    layers.Dense(10, activation='softmax')
])
m5_s.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['ac

ep_s = 2 if QUICK_DEMO else 10
print(f'Huấn luyện SVHN 3-Layer ({ep_s} epochs)...')
m3_s.fit(x_tr_s, y_tr_s, validation_split=0.1, epochs=ep_s, batch_size=128, verbose
print(f'\nHuấn luyện SVHN 5-Layer ({ep_s} epochs)...')
m5_s.fit(x_tr_s, y_tr_s, validation_split=0.1, epochs=ep_s, batch_size=128, verbose

_, acc3_s = m3_s.evaluate(x_te_s, y_te_s, verbose=0)
_, acc5_s = m5_s.evaluate(x_te_s, y_te_s, verbose=0)
print(f'\n[KẾT QUẢ TEST SVHN] 3-Layer: {acc3_s*100:.2f}% | 5-Layer: {acc5_s*100:.2f}
```

Huấn luyện SVHN 3-Layer (10 epochs)...

Epoch 1/10 - 35s - loss: 1.2973 - accuracy: 0.5851 - val_loss: 0.7339 - val_accuracy: 0.7903
Epoch 2/10 - 35s - loss: 0.6557 - accuracy: 0.8158 - val_loss: 0.6097 - val_accuracy: 0.8325
Epoch 3/10 - 35s - loss: 0.5857 - accuracy: 0.8384 - val_loss: 0.5934 - val_accuracy: 0.8393
Epoch 4/10 - 35s - loss: 0.5479 - accuracy: 0.8513 - val_loss: 0.5798 - val_accuracy: 0.8423
Epoch 5/10 - 35s - loss: 0.5198 - accuracy: 0.8598 - val_loss: 0.5615 - val_accuracy: 0.8453
Epoch 6/10 - 35s - loss: 0.4958 - accuracy: 0.8661 - val_loss: 0.5511 - val_accuracy: 0.8518
Epoch 7/10 - 35s - loss: 0.4746 - accuracy: 0.8708 - val_loss: 0.5381 - val_accuracy: 0.8554
Epoch 8/10 - 35s - loss: 0.4559 - accuracy: 0.8757 - val_loss: 0.5338 - val_accuracy: 0.8533
Epoch 9/10 - 35s - loss: 0.4382 - accuracy: 0.8804 - val_loss: 0.5276 - val_accuracy: 0.8549
Epoch 10/10 - 35s - loss: 0.4226 - accuracy: 0.8843 - val_loss: 0.5191 - val_accuracy: 0.8567

Huấn luyện SVHN 5-Layer (10 epochs)...

Epoch 1/10 - 255s - loss: 0.8921 - accuracy: 0.7282 - val_loss: 0.5338 - val_accuracy: 0.8397
Epoch 2/10 - 255s - loss: 0.4573 - accuracy: 0.8626 - val_loss: 0.3476 - val_accuracy: 0.8953
Epoch 3/10 - 255s - loss: 0.3850 - accuracy: 0.8849 - val_loss: 0.4072 - val_accuracy: 0.8780
Epoch 4/10 - 255s - loss: 0.3419 - accuracy: 0.8998 - val_loss: 0.3275 - val_accuracy: 0.9012
Epoch 5/10 - 255s - loss: 0.3077 - accuracy: 0.9086 - val_loss: 0.3631 - val_accuracy: 0.8894
Epoch 6/10 - 255s - loss: 0.2838 - accuracy: 0.9154 - val_loss: 0.2580 - val_accuracy: 0.9234
Epoch 7/10 - 255s - loss: 0.2645 - accuracy: 0.9215 - val_loss: 0.3729 - val_accuracy: 0.8928
Epoch 8/10 - 255s - loss: 0.2520 - accuracy: 0.9250 - val_loss: 0.2456 - val_accuracy: 0.9324
Epoch 9/10 - 255s - loss: 0.2381 - accuracy: 0.9285 - val_loss: 0.3062 - val_accuracy: 0.9066
Epoch 10/10 - 255s - loss: 0.2242 - accuracy: 0.9312 - val_loss: 0.2958 - val_accuracy: 0.9146

[KẾT QUẢ TEST SVHN] 3-Layer: 84.84% | 5-Layer: 91.94%

PHẦN C: HUẤN LUYỆN TRÊN BỘ DỮ LIỆU DIABETES (1D-CNN)

```
In [7]: # Cell 7: Bảng tổng kết 6 mô hình Keras
summary_k = pd.DataFrame([
    {'Dataset': 'Fashion-MNIST', 'Architecture': '3-Layer', 'Params': m3_f.count_pa
    {'Dataset': 'Fashion-MNIST', 'Architecture': '5-Layer', 'Params': m5_f.count_pa
    {'Dataset': 'SVHN', 'Architecture': '3-Layer', 'Params': m3_s.count_params(), '

```

```

{'Dataset': 'SVHN', 'Architecture': '5-Layer', 'Params': m5_s.count_params(), '
{'Dataset': 'Diabetes (1D)', 'Architecture': '3-Layer', 'Params': 7299, 'Test A
{'Dataset': 'Diabetes (1D)', 'Architecture': '5-Layer', 'Params': 43555, 'Test
])
display(summary_k)

```

	Dataset	Architecture	Params	Test Accuracy
0	Fashion-MNIST	3-Layer	50186	90.49%
1	Fashion-MNIST	5-Layer	469098	92.32%
2	SVHN	3-Layer	60362	84.84%
3	SVHN	5-Layer	592554	91.94%
4	Diabetes (1D)	3-Layer	7299	69.25%
5	Diabetes (1D)	5-Layer	43555	68.62%